

MemoLink Deployment Guide

Backend: AWS Lambda (container image) + API Gateway

Frontend: Azure Static Web Apps

Database: Supabase (unchanged)

Overview

GitHub push to main

- └─ backend files changed → GitHub Actions → ECR → Lambda
- └─ frontend files changed → GitHub Actions → Azure Static Web Apps

Part 1 — AWS Setup

1.1 Create an IAM User for GitHub Actions

GitHub Actions needs AWS credentials to push to ECR and update Lambda.

1. Go to **AWS Console** → **IAM** → **Users** → **Create user**
2. Name it `memolink-github-actions`
3. Select **"Attach policies directly"**
4. Attach these two managed policies:
 - `AmazonEC2ContainerRegistryPowerUser`
 - `AWSLambdaRole`
5. Click **Create user**
6. Open the user → **Security credentials** tab → **Create access key**
7. Choose **"Application running outside AWS"**
8. Copy the values — you only see the secret once:

```
AWS_ACCESS_KEY_ID      = AKIA...  
AWS_SECRET_ACCESS_KEY = wJalr...
```

If you need a tighter permission policy instead of managed policies, the minimum actions required are:

```
ecr:GetAuthorizationToken, ecr:BatchCheckLayerAvailability,  
ecr:GetDownloadUrlForLayer, ecr:PutImage, ecr:InitiateLayerUpload,  
ecr:UploadLayerPart, ecr:CompleteLayerUpload, lambda:UpdateFunctionCode,  
lambda:GetFunction, lambda:GetFunctionConfiguration
```

1.2 Create an ECR Repository

ECR stores the Docker image that Lambda runs.

1. Go to **AWS Console** → **ECR** → **Create repository**
2. Set visibility to **Private**
3. Name it **memolink-backend**
4. Leave all other settings as default → **Create repository**

```
ECR_REPOSITORY = memolink-backend
AWS_REGION     = (the region you created it in, e.g. ap-southeast-2)
```

1.3 Push the First Image Manually

Lambda needs an image in ECR before you can create the function. Run these commands locally once.

```
# Authenticate Docker to ECR (replace ACCOUNT_ID and REGION)
aws ecr get-login-password --region REGION | \
  docker login --username AWS --password-stdin
ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com

# Build and push
docker build -t memolink-backend .
docker tag memolink-backend:latest
ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/memolink-backend:latest
docker push ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/memolink-backend:latest
```

1.4 Create the Lambda Function

1. Go to **AWS Console** → **Lambda** → **Create function**
2. Choose **"Container image"**
3. Function name: **memolink-api**
4. Click **"Browse images"** → select the ECR repo → select the **latest** image
5. Architecture: **x86_64**
6. Click **Create function**

After creation, open the function and change these settings under **Configuration**:

Setting	Value
Timeout	120 seconds
Memory	512 MB
Ephemeral storage	512 MB

```
LAMBDA_FUNCTION_NAME = memolink-api
```

1.5 Set Lambda Environment Variables

Inside the Lambda function → **Configuration** → **Environment variables** → **Edit**, add every variable from your `.env` file:

Key	Value
<code>DATABASE_URL</code>	Your Supabase connection string
<code>OPENAI_API_KEY</code>	<code>sk-...</code>
<code>OPENAI_CHAT_MODEL</code>	<code>gpt-4o-mini</code>
<code>OPENAI_EMBEDDING_MODEL</code>	<code>text-embedding-3-small</code>
<code>GEMINI_API_KEY</code>	Your Gemini key
<code>DEEPSEEK_API_KEY</code>	Your DeepSeek key (optional)
<code>BRAVE_SEARCH_API_KEY</code>	Your Brave key
<code>SEMANTIC_SCHOLAR_API_KEY</code>	Optional
<code>JWT_SECRET_KEY</code>	Long random string (32+ chars)
<code>JWT_ALGORITHM</code>	<code>HS256</code>
<code>SMTP_HOST</code>	<code>smtp.gmail.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>SMTP_USER</code>	Your Gmail address
<code>SMTP_PASSWORD</code>	Your Gmail app password
<code>SMTP_FROM</code>	Your Gmail address
<code>FRONTEND_URL</code>	Your Azure Static Web App URL (after Part 2)

JWT_SECRET_KEY tip: Generate one with `python -c "import secrets; print(secrets.token_hex(32))"`

1.6 Create an API Gateway (HTTP API)

Lambda needs an HTTP endpoint so the frontend can reach it.

1. Go to **AWS Console** → **API Gateway** → **Create API**
2. Choose **HTTP API** → **Build**
3. Add integration: **Lambda** → select `memolink-api`
4. API name: `memolink-api-gateway`
5. Configure routes: set route to `$default` (catch-all)
6. Stage name: `$default` (auto-deploy enabled)
7. Click **Create**

After creation, copy the **Invoke URL** shown on the API page. It looks like:

```
https://xxxxxxxxxx.execute-api.ap-southeast-2.amazonaws.com
```

This becomes `VITE_API_BASE_URL` in GitHub secrets.

Enable CORS on the API Gateway:

1. Open your API → **CORS**
2. Set:
 - Access-Control-Allow-Origin: `*` (or your Azure URL after Part 2)
 - Access-Control-Allow-Methods: `*`
 - Access-Control-Allow-Headers: `*`
3. Save

Part 2 — Azure Static Web Apps Setup

2.1 Create the Static Web App

1. Go to **portal.azure.com** → **Create a resource** → **Static Web App**
2. Fill in:
 - Subscription: your subscription
 - Resource group: create new → `memolink-rg`
 - Name: `memolink-web`
 - Plan type: **Free**
 - Region: closest to your users
 - Deployment source: **Other** (we deploy via GitHub Actions manually)
3. Click **Review + create** → **Create**

2.2 Get the Deployment Token

1. Open the Static Web App resource → **Manage deployment token**
2. Copy the token

```
AZURE_STATIC_WEB_APPS_API_TOKEN = <the token you copied>
```

2.3 Note Your Azure URL

After creation the URL is shown on the overview page:

```
https://memolink-web.azurestaticapps.net (example)
```

Use this as `FRONTEND_URL` in the Lambda environment variables (Part 1.5).

Part 3 — GitHub Actions Secrets

Go to your GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

Add each secret below exactly as named (case-sensitive):

Backend secrets

Secret name	Where to get it
<code>AWS_ACCESS_KEY_ID</code>	Part 1.1 — IAM user access key
<code>AWS_SECRET_ACCESS_KEY</code>	Part 1.1 — IAM user secret key
<code>AWS_REGION</code>	The AWS region you used (e.g. <code>ap-southeast-2</code>)
<code>ECR_REPOSITORY</code>	<code>memolink-backend</code> (the ECR repo name from Part 1.2)
<code>LAMBDA_FUNCTION_NAME</code>	<code>memolink-api</code> (the Lambda function name from Part 1.4)

Frontend secrets

Secret name	Where to get it
<code>AZURE_STATIC_WEB_APPS_API_TOKEN</code>	Part 2.2 — Azure deployment token
<code>VITE_API_BASE_URL</code>	Part 1.6 — API Gateway Invoke URL (no trailing slash)

Part 4 — First Deployment

Once all secrets are set, trigger CI/CD by pushing to `main`:

```
git push origin main
```

Go to your repo → **Actions** tab to watch the workflows run.

- `Deploy Backend to AWS Lambda` — builds the Docker image, pushes to ECR, updates Lambda (~3-5 min)
- `Deploy Frontend to Azure Static Web Apps` — builds Vite, deploys to Azure (~2 min)

Part 5 — Verify

Check the backend

```
GET https://xxxxxxxxxx.execute-api.ap-southeast-2.amazonaws.com/api/health
```

Expected response:

```
{ "status": "ok", "service": "MemoLink API" }
```

Check the frontend

Open your Azure URL in a browser. You should see the MemoLink login page.

Known Limitations on Lambda

Feature	Behaviour
AI chat streaming	Disabled — full answer appears at once after generation completes
Research Mode	Works but may be slow on complex queries (120s timeout)
All other features	Work normally

Chat streaming is buffered because API Gateway does not support Server-Sent Events. The answer is still correct — it just appears all at once instead of token-by-token.

Secrets Quick Reference

```
# GitHub Actions secrets (repo → Settings → Secrets → Actions)

AWS_ACCESS_KEY_ID           IAM user key
AWS_SECRET_ACCESS_KEY       IAM user secret
AWS_REGION                   e.g. ap-southeast-2
ECR_REPOSITORY               memolink-backend
LAMBDA_FUNCTION_NAME         memolink-api
AZURE_STATIC_WEB_APPS_API_TOKEN  from Azure portal
VITE_API_BASE_URL            API Gateway invoke URL

# Lambda environment variables (set in Lambda console, not GitHub)

DATABASE_URL
OPENAI_API_KEY
OPENAI_CHAT_MODEL
OPENAI_EMBEDDING_MODEL
GEMINI_API_KEY
DEEPSEEK_API_KEY
BRAVE_SEARCH_API_KEY
SEMANTIC_SCHOLAR_API_KEY
JWT_SECRET_KEY
JWT_ALGORITHM
SMTP_HOST
SMTP_PORT
SMTP_USER
SMTP_PASSWORD
```

```
SMTP_FROM  
FRONTEND_URL
```